De-Duplication Using Locality Sensitive Hashing

Bhisham (D18039) & Himanshu (V18011)

Programming Practicum (CS 571)

December 2, 2019

Scaling the Heights

IIT Mandi

Indian Institute of Technology Mandi

Problem Statement and Applications

- **Problem Statement**

  The aim is to implement de duplication model to find the duplicate documents from the large collection of documents via clustering using locality sensitive hashing and prune the duplicate documents.

Problem Statement and Applications

- **Problem Statement**
  The aim is to implement de duplication model to find the duplicate documents from the large collection of documents via clustering using locality sensitive hashing and prune the duplicate documents.
- **Applications:**

Problem Statement and Applications

- **Problem Statement**
  The aim is to implement de duplication model to find the duplicate documents from the large collection of documents via clustering using locality sensitive hashing and prune the duplicate documents.
- **Applications:**
  - Filter duplicate documents in search engine

Problem Statement and Applications

- **Problem Statement**
  The aim is to implement de duplication model to find the duplicate documents from the large collection of documents via clustering using locality sensitive hashing and prune the duplicate documents.
- **Applications:**
  - Filter duplicate documents in search engine
  - Plagiarism, mirror pages

Problem Statement and Applications

- **Problem Statement**
  The aim is to implement de duplication model to find the duplicate documents from the large collection of documents via clustering using locality sensitive hashing and prune the duplicate documents.
- **Applications:**
  - Filter duplicate documents in search engine
  - Plagiarism, mirror pages
  - Recommend similar products, movies etc.

Challenges In Du-duplication

- Many small pieces of one document can appear out of order in another.

Challenges In Du-duplication

- Many small pieces of one document can appear out of order in another.
- High dimensionality of the unique characteristic/feature vector required to identify documents.

Challenges In Du-duplication

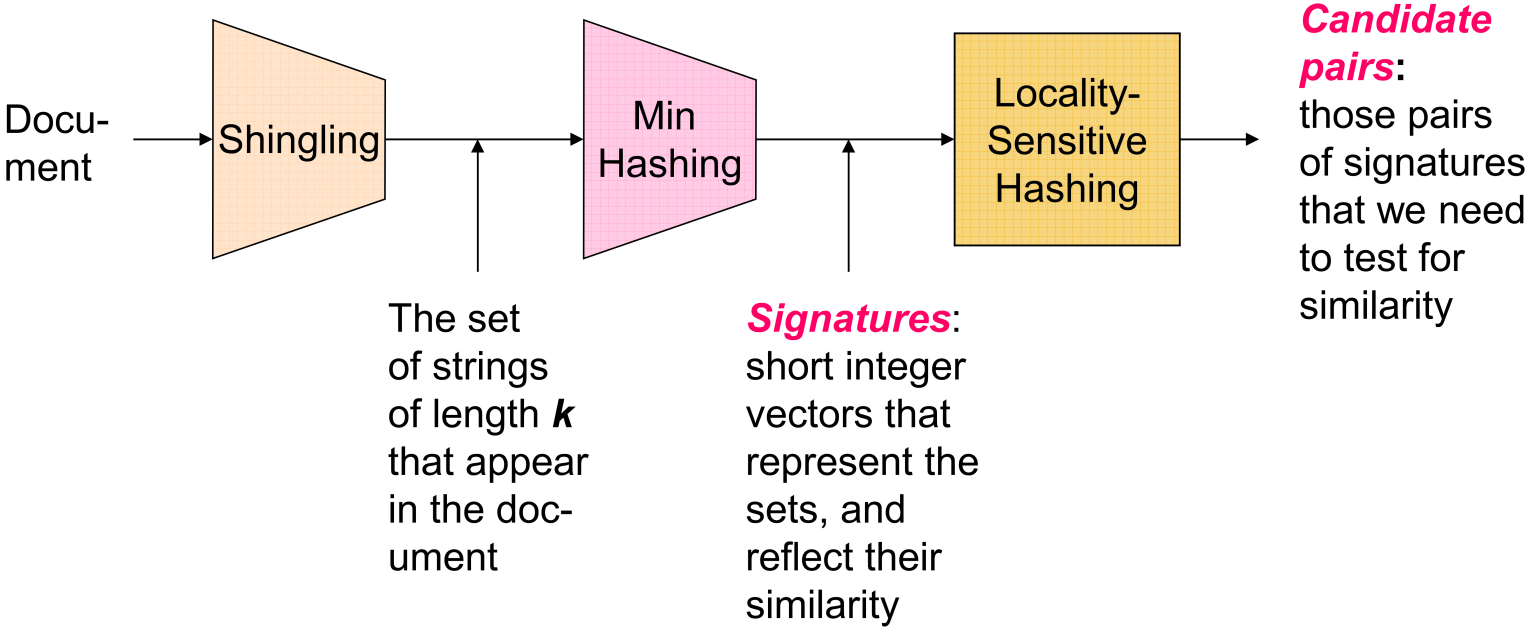
- Many small pieces of one document can appear out of order in another.
- High dimensionality of the unique characteristic/feature vector required to identify documents.
- Documents can be very large or so many, may not possible to fit in main memory.

Challenges In Du-duplication

- Many small pieces of one document can appear out of order in another.
- High dimensionality of the unique characteristic/feature vector required to identify documents.
- Documents can be very large or so many, may not possible to fit in main memory.
- In case of too many documents(say 1 billion) to compute pair wise similarity for each pair of document is highly expansive and time taking.

Big Picture of De-duplication Model

Big Picture of De-duplication Model



Source: J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets

Jaccard Similarity

- **Jaccard Similarity:** Let S1 and S2 are two sets, then jaccard similarity between two sets is defined as

Jaccard Similarity

- **Jaccard Similarity:** Let S1 and S2 are two sets, then jaccard similarity between two sets is defined as

Jaccard Similarity

- **Jaccard Similarity:** Let S1 and S2 are two sets, then jaccard similarity between two sets is defined as

$$\text{SIM(S1,S2)} = \frac{|S1 \cap S2|}{|S1 \cup S2|}$$

Shingling

- **Shingling:** Shingling convert documents to sets.

Shingling

- **Shingling:** Shingling convert documents to sets.
- **k-shingle:** A k-shingle is consecutive sub-string of k-words(or k-characters) of the document string. The alternate name for k-shingle is k-gram.

Shingling

- **Shingling:** Shingling convert documents to sets.
- **k-shingle:** A k-shingle is consecutive sub-string of k-words(or k-characters) of the document string. The alternate name for k-shingle is k-gram.
- **Example:** If $D1$ and $D2$ two sets,

Shingling

- **Shingling:** Shingling convert documents to sets.
- **k-shingle:** A k-shingle is consecutive sub-string of k-words(or k-characters) of the document string. The alternate name for k-shingle is k-gram.
- **Example:** If $D1$ and $D2$ two sets,

  $\boldsymbol{D1}$ **:** De duplication Using Locality Sensitive Hashing

Shingling

- **Shingling:** Shingling convert documents to sets.
- **k-shingle:** A k-shingle is consecutive sub-string of k-words(or k-characters) of the document string. The alternate name for k-shingle is k-gram.
- **Example:** If $D1$ and $D2$ two sets,

  $\boldsymbol{D1}$ **:** De duplication Using Locality Sensitive Hashing

  $\boldsymbol{D2}$ **:** Using Locality Sensitive Hashing for De duplication

Shingling

- $\boldsymbol{D1}$ = {"De duplication", "duplication Using", "Using Locality", "Locality Sensitive", "Sensitive Hashing"}

Shingling

- $\boldsymbol{D1}$ = {“De duplication”, “duplication Using”, “Using Locality”, “Locality Sensitive”, “Sensitive Hashing”}
- $\boldsymbol{D2}$ = {“Using Locality”, “Locality Sensitive”, “Sensitive Hashing”, “Hashing for”, “for De”, “De duplication”}

Shingling

- $\boldsymbol{D1}$ = {"De duplication", "duplication Using", "Using Locality", "Locality Sensitive", "Sensitive Hashing"}
- $\boldsymbol{D2}$ = {"Using Locality", "Locality Sensitive", "Sensitive Hashing", "Hashing for", "for De", "De duplication"}
- $\boldsymbol{D1} \cap \boldsymbol{D2}$ = {"De duplication', "Using Locality", "Locality Sensitive", "Sensitive Hashing"}

Shingling

- $\boldsymbol{D1}$ = {"De duplication", "duplication Using", "Using Locality", "Locality Sensitive", "Sensitive Hashing"}
- $\boldsymbol{D2}$ = {"Using Locality", "Locality Sensitive", "Sensitive Hashing", "Hashing for", "for De", "De duplication"}
- $\boldsymbol{D1} \cap \boldsymbol{D2}$ = {"De duplication', "Using Locality", "Locality Sensitive", "Sensitive Hashing"}
- $\boldsymbol{D1} \cup \boldsymbol{D2}$ = {"De duplication", "duplication Using", "Using Locality", "Locality Sensitive", "Sensitive Hashing", "Hashing for", "for De"}

Shingling

- $\boldsymbol{D1}$ = {"De duplication", "duplication Using", "Using Locality", "Locality Sensitive", "Sensitive Hashing"}
- $\boldsymbol{D2}$ = {"Using Locality", "Locality Sensitive", "Sensitive Hashing", "Hashing for", "for De", "De duplication"}
- $\boldsymbol{D1 \cap D2}$ = {"De duplication', "Using Locality", "Locality Sensitive", "Sensitive Hashing"}
- $\boldsymbol{D1 \cup D2}$ = {"De duplication", "duplication Using", "Using Locality", "Locality Sensitive", "Sensitive Hashing", "Hashing for", "for De"}

$$\boldsymbol{SIM(D1, D2) = \frac{|D1 \cap D2|}{|D1 \cup D2|} = \frac{4}{7} = 0.57}$$

Characteristic Matrix Corresponding to Documents

- For above two document sets $D1$ and $D2$ the characteristic matrix is as follow

Characteristic Matrix Corresponding to Documents

- For above two document sets $D1$ and $D2$ the characteristic matrix is as follow

Characteristic Matrix Corresponding to Documents

- For above two document sets $D1$ and $D2$ the characteristic matrix is as follow

$D1 \cup D2$	$D1$	$D2$
“De duplication"	1	1
“duplication Using"	1	0
“Using Locality"	1	1
“Locality Sensitive"	1	1
“Sensitive Hashing"	1	1
“Hashing for"	0	1
“for De"	0	1

Min-Hashing

- The aim of min-hashing is to convert large sets to short signatures, while preserving similarity.

Min-Hashing

- The aim of min-hashing is to convert large sets to short signatures, while preserving similarity.
- Suppose the rows of the boolean matrix permuted under random permutation π

Min-Hashing

- The aim of min-hashing is to convert large sets to short signatures, while preserving similarity.
- Suppose the rows of the boolean matrix permuted under random permutation $\boldsymbol{\pi}$
- Define a **hash** function $\boldsymbol{h_\pi(C)}$ = the index of the first (in the permuted order $\boldsymbol{\pi}$) row in which column $\boldsymbol{C}$ has value $\mathbf{1}$:

$$\boldsymbol{h_\pi(C) = min_\pi \pi(C)}$$

Example of Signature Matrix

Example of Signature Matrix

Permutation π | Input matrix (Shingles x Documents)

2	4	3	1	0	1	0
3	2	4	1	0	0	1
7	1	7	0	1	0	1
6	3	2	0	1	0	1
1	6	6	0	1	0	1
5	7	1	1	0	1	0
4	5	5	1	0	1	0

Signature matrix *M*

2	1	2	1
2	1	4	1
1	2	1	2

Similarities:

	1-3	2-4	1-2	3-4
Col/Col	0.75	0.75	0	0
Sig/Sig	0.67	1.00	0	0

Source: J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets

Locality Sensitive Hashing

- The basic idea of locality sensitive hashing to use a function f(x,y) that tells whether x and y is a candidate pair(i.e. a pair of elements whose similarity must be evaluated).

Locality Sensitive Hashing

- The basic idea of locality sensitive hashing to use a function f(x,y) that tells whether x and y is a candidate pair(i.e. a pair of elements whose similarity must be evaluated).
- **Method to find such f from Min-hash Signature matrix:**

Locality Sensitive Hashing

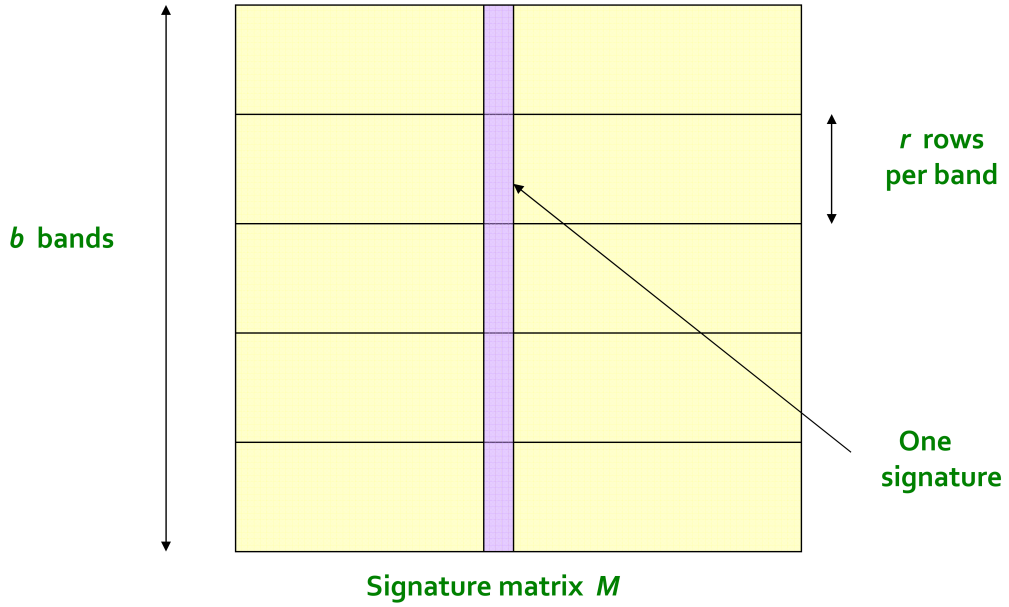
- The basic idea of locality sensitive hashing to use a function f(x,y) that tells whether x and y is a candidate pair(i.e. a pair of elements whose similarity must be evaluated).
- **Method to find such f from Min-hash Signature matrix:**
  - Divide the rows of signature matrix in b band having r rows.

Locality Sensitive Hashing

- The basic idea of locality sensitive hashing to use a function f(x,y) that tells whether x and y is a candidate pair(i.e. a pair of elements whose similarity must be evaluated).
- **Method to find such f from Min-hash Signature matrix:**
  - Divide the rows of signature matrix in b band having r rows.

Locality Sensitive Hashing

- The basic idea of locality sensitive hashing to use a function f(x,y) that tells whether x and y is a candidate pair(i.e. a pair of elements whose similarity must be evaluated).
- **Method to find such f from Min-hash Signature matrix:**
  - Divide the rows of signature matrix in b band having r rows.



Source: J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets

Locality Sensitive Hashing

Locality Sensitive Hashing

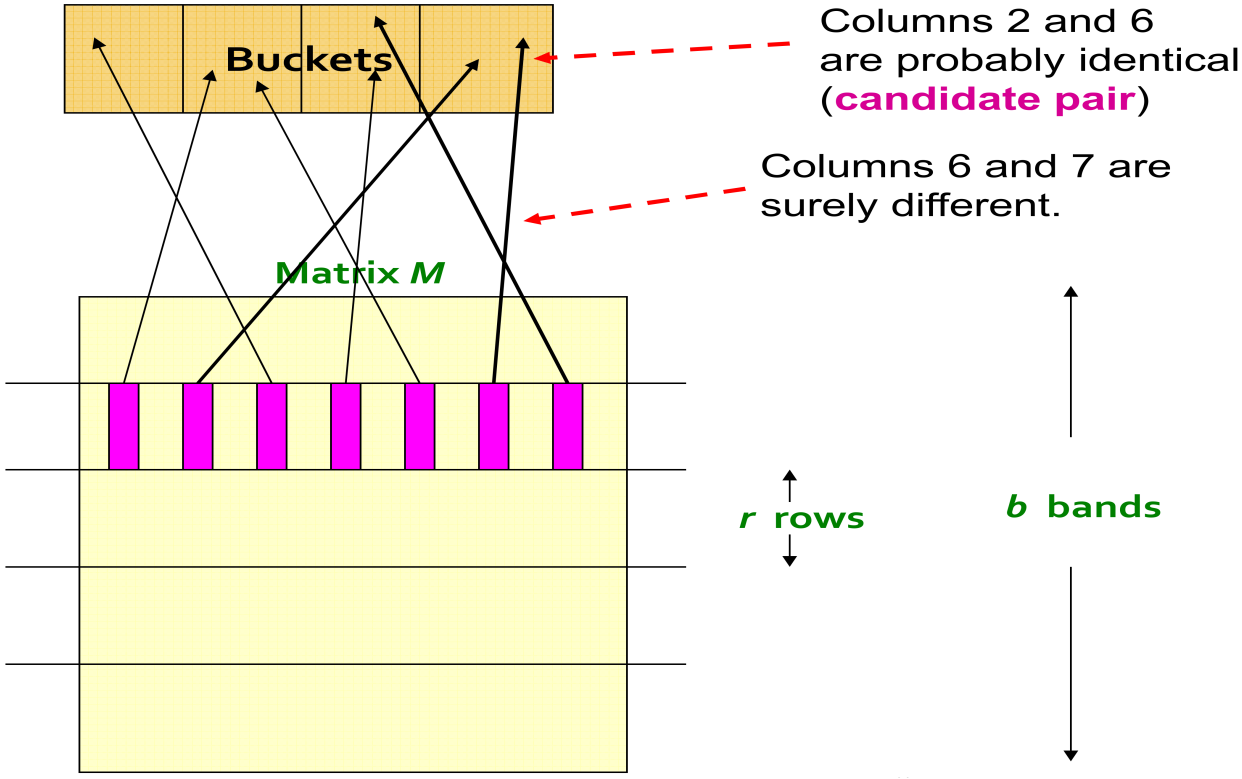
- For each band hash the column in k buckets, buckets contains column pairs if there are identical in particular band and the pair of a bucket are called **candidate pairs**

Locality Sensitive Hashing

- For each band hash the column in k buckets, buckets contains column pairs if there are identical in particular band and the pair of a bucket are called **candidate pairs**

Locality Sensitive Hashing

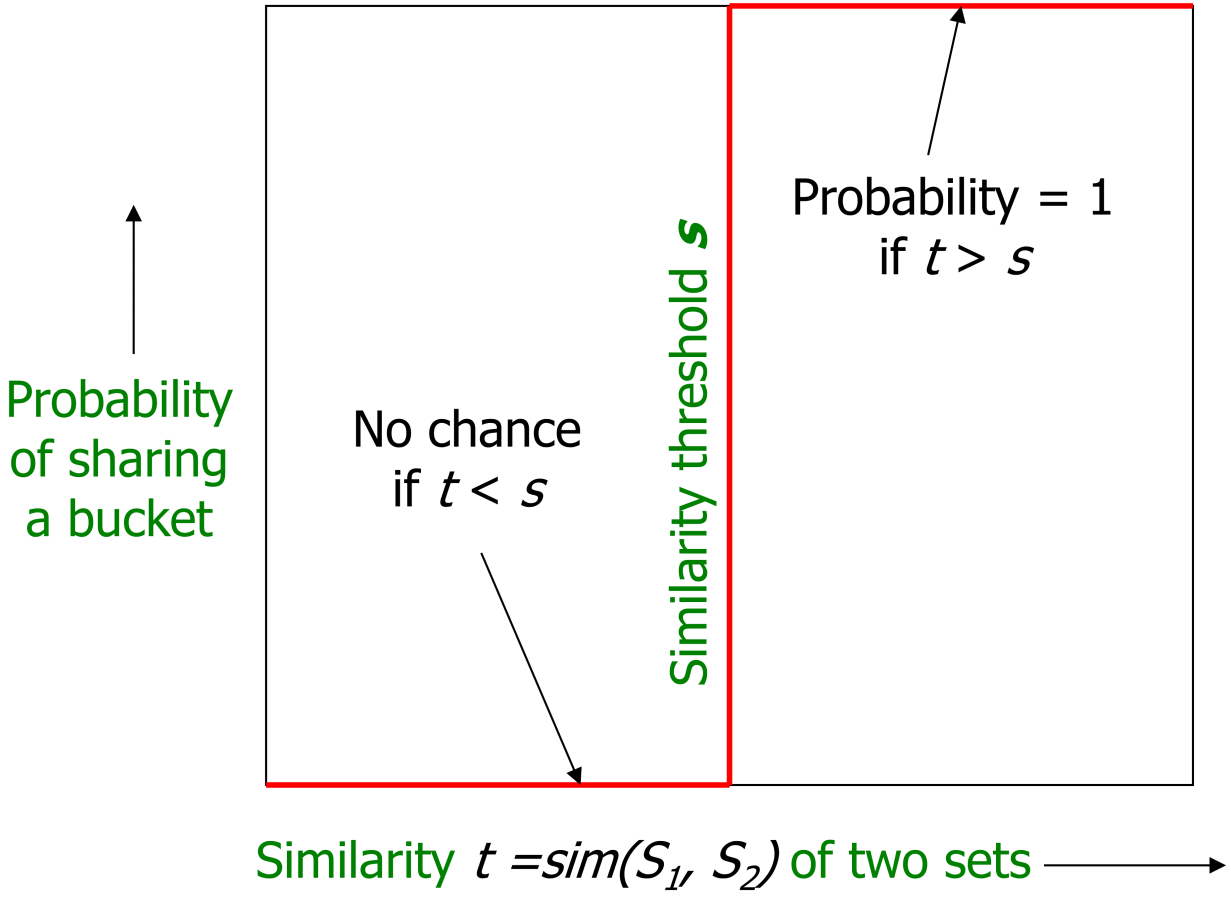
- For each band hash the column in k buckets, buckets contains column pairs if there are identical in particular band and the pair of a bucket are called **candidate pairs**



Source: J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets

Interpretation of b bands for r rows:

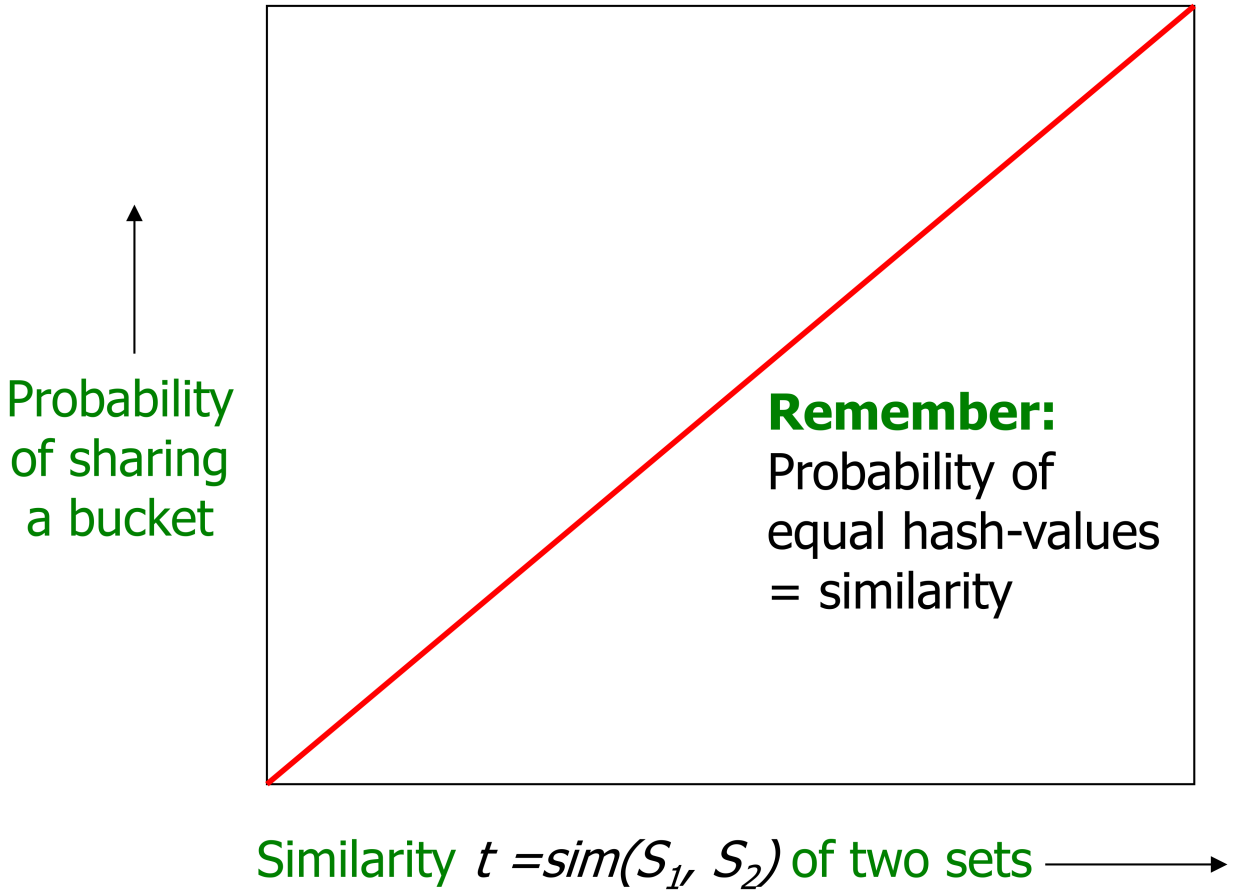
- We want that for column whose similarity is greater than threshold t will goes to same bucket with probability 1 .



Source: J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets

Interpretation of b bands for r rows:

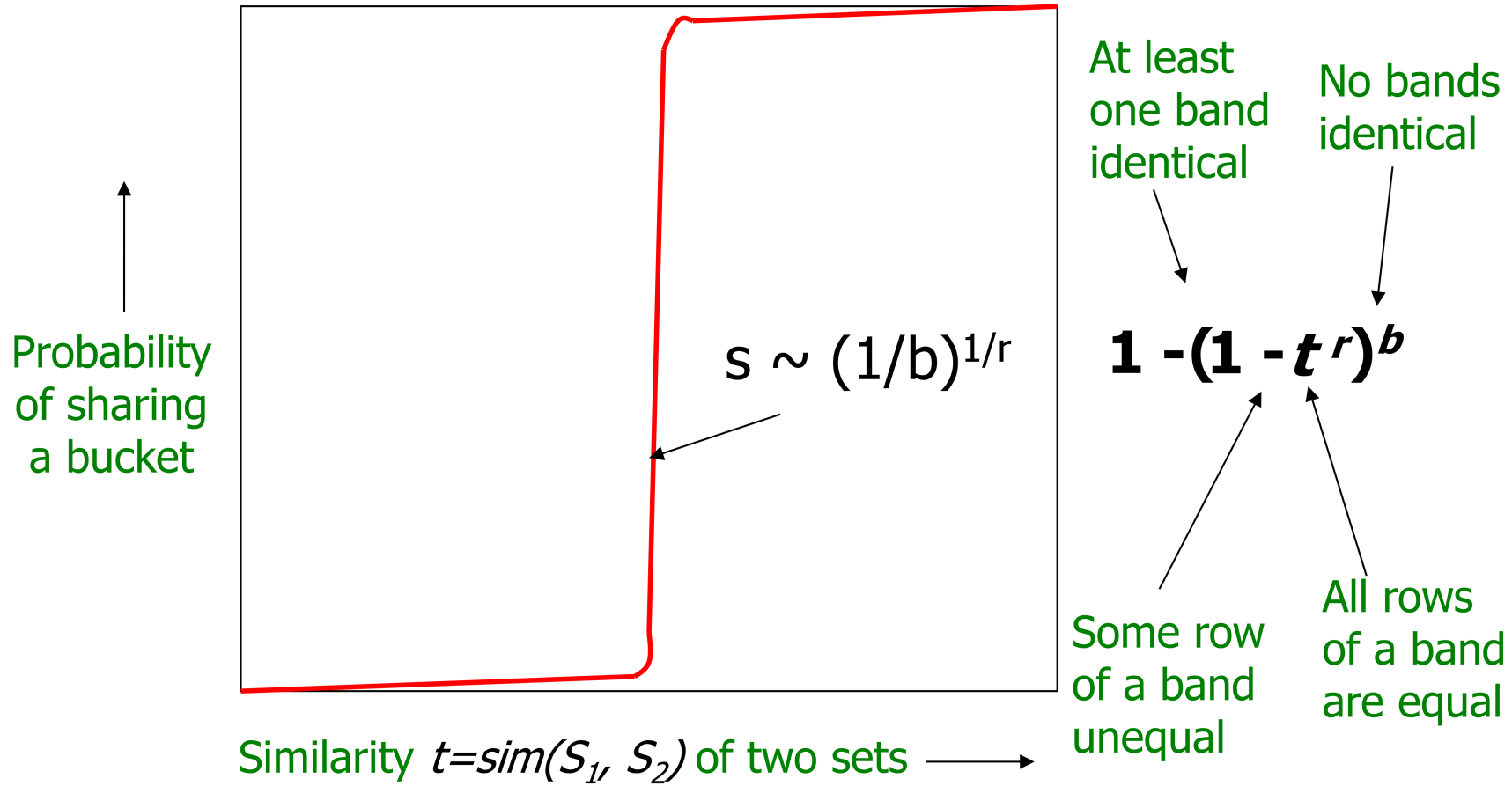
- For $b = 1$ and $r = 1$:



Source: J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets

Interpretation of b bands for r rows:

- For b bands and r rows:



Source: J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets

Results

- To check the Working and performance of the implemented locality sensitive hashing algorithm, **20 Newsgroup** text data set containing 600 hundred text files is taken in consideration.

Results

- To check the Working and performance of the implemented locality sensitive hashing algorithm, **20 Newsgroup** text data set containing 600 hundred text files is taken in consideration.
- The actual truth about the data set is not known, we computed the jaccard similarity of all combinations of text files, and those having jaccard similarity greater than 0.55, kept in same cluster.

Results

- To check the Working and performance of the implemented locality sensitive hashing algorithm, **20 Newsgroup** text data set containing 600 hundred text files is taken in consideration.
- The actual truth about the data set is not known, we computed the jaccard similarity of all combinations of text files, and those having jaccard similarity greater than 0.55, kept in same cluster.
- We found 8 such clusters and are as follow:

Results

- To check the Working and performance of the implemented locality sensitive hashing algorithm, **20 Newsgroup** text data set containing 600 hundred text files is taken in consideration.
- The actual truth about the data set is not known, we computed the jaccard similarity of all combinations of text files, and those having jaccard similarity greater than 0.55, kept in same cluster.
- We found 8 such clusters and are as follow:

Results

- To check the Working and performance of the implemented locality sensitive hashing algorithm, **20 Newsgroup** text data set containing 600 hundred text files is taken in consideration.
- The actual truth about the data set is not known, we computed the jaccard similarity of all combinations of text files, and those having jaccard similarity greater than 0.55, kept in same cluster.
- We found 8 such clusters and are as follow:

Table: Actual Truth

Cluster Number	File ID's
1	**[51150, 51240]**
2	**[51270, 51307]**
3	**[53072, 53178]**
4	**[37917, 37950]**
5	**[38216, 38255]**
6	**[38227, 38330]**
7	**[38320, 38432]**
8	**[38438, 38442, 38458]**

Results

- k (for k-shingle) = 5

Results

- k (for k-shingle) = 5
- Total number of hash functions for Min-hashing = 100

Results

- k (for k-shingle) = 5
- Total number of hash functions for Min-hashing = 100
- Number of bands (b) = 10

Results

- k (for k-shingle) = 5
- Total number of hash functions for Min-hashing = 100
- Number of bands (b) = 10
- Number of rows in a band (r) = 6

Results

- k (for k-shingle) = 5
- Total number of hash functions for Min-hashing = 100
- Number of bands (b) = 10
- Number of rows in a band (r) = 6

Results

- k (for k-shingle) = 5
- Total number of hash functions for Min-hashing = 100
- Number of bands (b) = 10
- Number of rows in a band (r) = 6

Table: Predicted Duplicates

Cluster Number	File ID's
1	[38227, 38330]
2	[37917, 37950]
3	[38354, 38366]
4	[38435, 38457]
5	[53117, 53134]
6	[53174, 53195]
7	[38438, 38442, 38458]
8	[51186, 51210]
9	[38396, 38461]
10	[53072, 53178]

Results

- On running the LSH for same setting we get

Results

- On running the LSH for same setting we get

Results

- On running the LSH for same setting we get

Table: Predicted Duplicates

Cluster Number	File ID's
1	[51150, 51240]
2	[53174, 53195]
3	[37941, 37960]
4	[38320, 38432]
5	[53072, 53178]
6	[38442, 38458]
7	[51197, 51198]
8	[53064, 38099]
9	[38217, 38330]
10	[51290, 51304]
11	[51197, 51258]
12	[38217, 38227]
13	[51194, 51277]
14	[38216, 38255]
15	[53173, 53191]
16	[38217, 38227, 38330]

References I

Rajaraman, Anand, and Jeffrey David Ullman. Mining of massive datasets. Cambridge University Press, 2011.

Thank you